

MACARENA BENGUIGUI

Portfolio · Unity Developer · Technical Art & Tools

A selection of work where I got art running in real projects: tools I built, shader work, asset pipelines and before/after results.

mbenguiguicamara@gmail.com
linkedin.com/in/macarenabenguigui
+34 606 835 445 · Madrid, Spain · remote

CONTENTS

01 Ping Pong Fury
Personal project · Unity · 2D UI, triplanar shader

02 2D asset pipeline and level editor
Arcade game · web (Phaser) · cut-outs, prop catalogue, levels

03 Editor tools and mobile performance
AR app · Unity · texture automation, localization checks

04 3D anatomy viewer
Unity WebGL · UI, blurred pop-ups, section tool

05 Shipped apps
App Store and Google Play

01 · PERSONAL PROJECT

Ping Pong Fury

Unity 2022.3 · C# · built end to end as a technical test, with AI-assisted coding

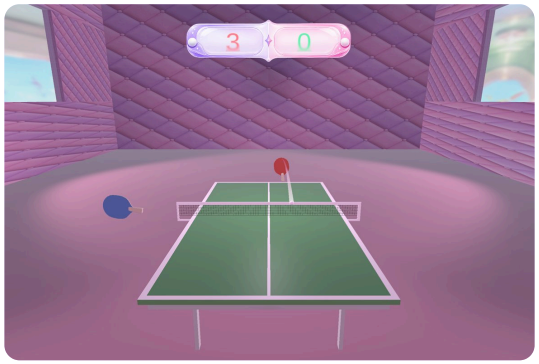
- **Full game:** 3D table tennis against an AI with three difficulty levels, official serve rules and hand-tuned ball physics.
- **2D UI in Unity:** pastel menu, difficulty buttons with mascots, score HUD, staggered entrance animations, 9-slice sprites for buttons, glows and shadows.
- **AI-generated art, integrated:** logo, mascots, background and HUD frame generated, cleaned and set up as sprites.
- **Clean delivery:** removed paid plugins (DOTween replaced with coroutines), reviewed the architecture and documented the code.

SHADER: TRIPLANARWALL

The quilted wall texture stretched and broke at every seam because each wall had a different scale. I wrote a triplanar shader that projects the texture in world space at a fixed size, so all 11 wall pieces line up with no UV work.



BEFORE · STANDARD MATERIAL



AFTER · TRIPLANAR SHADER



Side walls on the left: with the standard material the diamonds stretch and do not line up; with the triplanar shader they keep the same size everywhere.

02 · CLIENT PROJECT

2D asset pipeline and level editor

Arcade training game · web (Phaser) · Node scripts

- **Cut-outs:** batch removal of chroma-key backgrounds and leftover fringes, plus an audit that flags bad alpha and loose fragments on a checkerboard contact sheet.
- **Tiles and props:** seamless tile edges matched pixel by pixel, a unified tileset sheet for the whole prop catalogue and colliders generated in batch.
- **Level editor:** visual editor built on that catalogue so the training team can compose and update levels without a developer.
- **Level design:** levels laid out with the editor, plus a script that builds a full level procedurally from about 400 pieces.

Built with AI-assisted development; I defined the rules each script checks and reviewed the output.

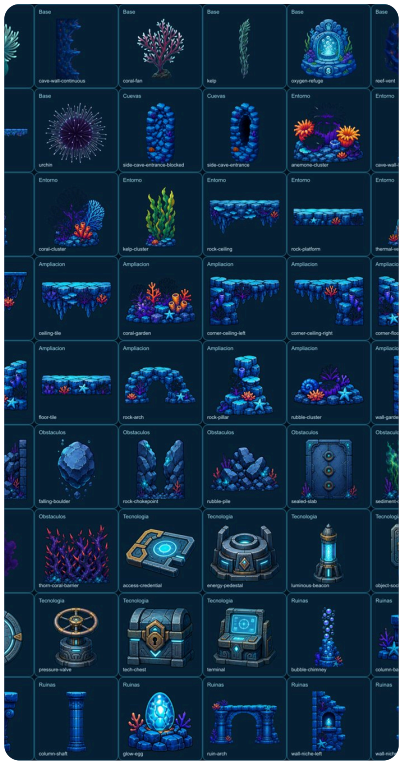
RAW EXPORT



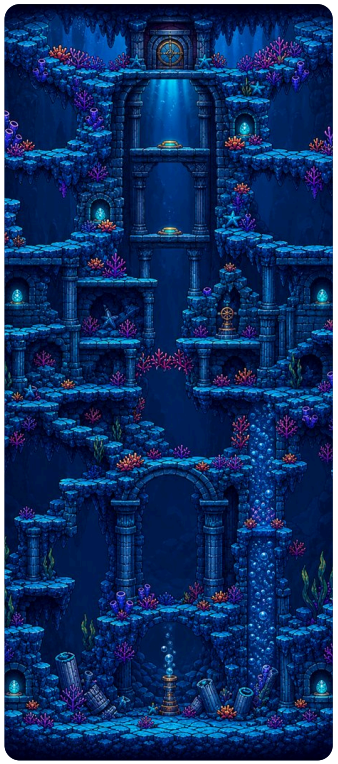
CLEAN CUT-OUT



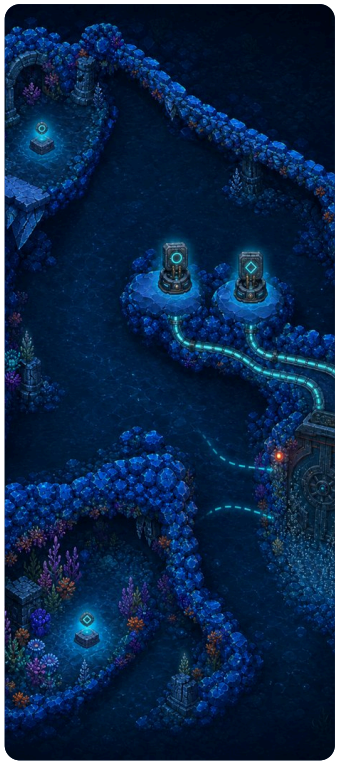
PROP CATALOGUE



LEVEL 1



LEVEL 3



Levels composed in the editor from the cleaned prop catalogue.

03 · CLIENT PROJECT

Editor tools and mobile performance

AR wildlife app · Unity 6 · C# editor scripting · iOS and Android

TEXTURE AUTOMATION

BEFORE
415 model textures at 2048 px, no mobile override, no mipmaps.

→

AFTER
1024 px max on Android and iOS, ASTC 6×6 (normals 5×5), mipmaps on.

- **MobileTextureOptimizer:** one menu click applies the platform overrides; the source PNGs are not touched, so it is reversible.
- **TextureSizeReport:** real imported size and memory of every texture, and whether a build uses it. It found 82 MB of habitat textures that never reached the build.

LOCALIZATION PIPELINE

BEFORE
Client texts edited in Excel and copied into the game by hand; overflowing text found on device.

→

AFTER
Python script syncs the Excel into the game; an editor report lists every text that does not fit its box.

- **Excel sync:** audit mode first, then apply, with a backup; documented so anyone in the team can run it.
- **LocalizationOverflowReport:** measures every localized text in Spanish, English and German against its UI box, without touching the scenes.
- **Validators:** catch stray overrides in prefab variants and unapplied material changes before a build.

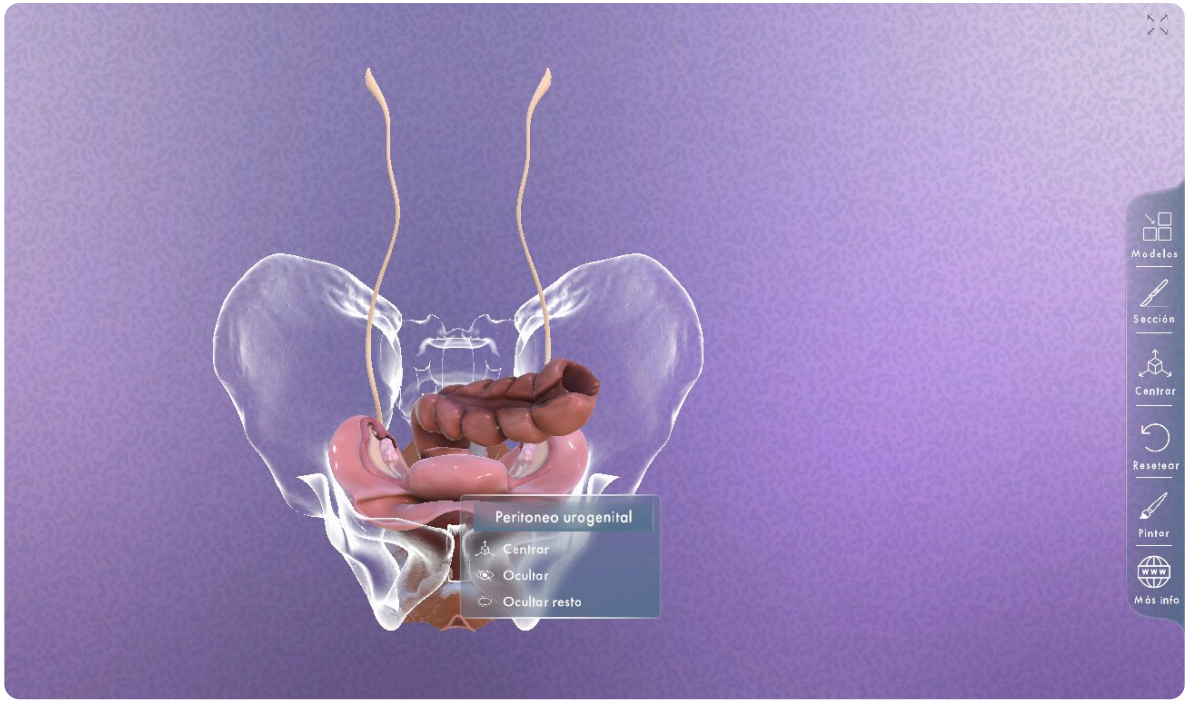
All tools written with AI-assisted development (Claude Code). I decide what each tool must check, then review and test it in the real project before the team uses it. Documented in the project's developer manual.

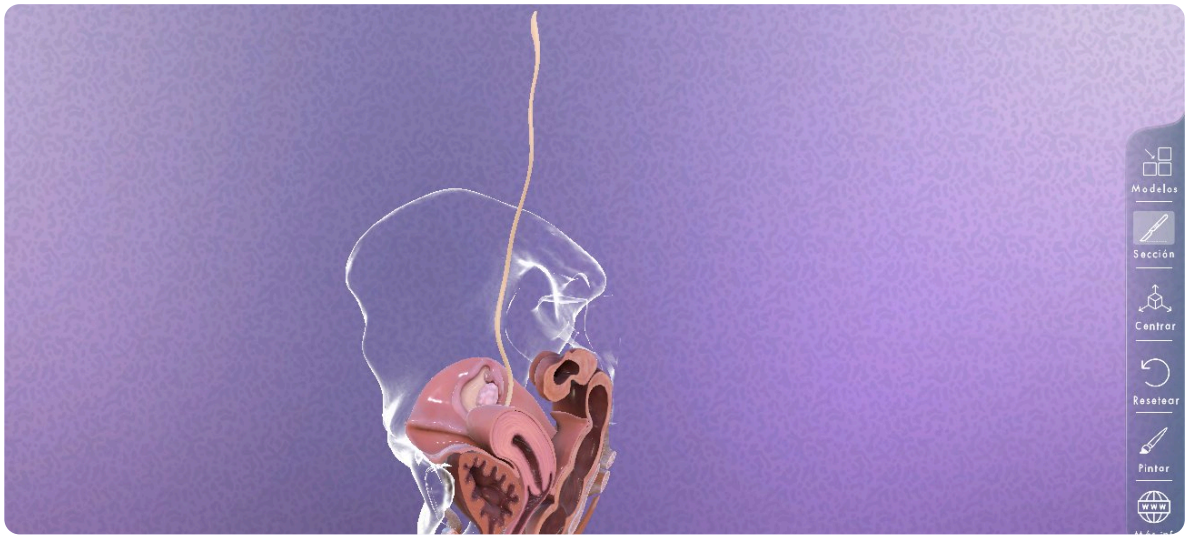
04 · CLIENT PROJECT

3D anatomy viewer

Unity WebGL · C# · UI

- **Whole viewer built in Unity WebGL:** camera, selection, toolbar, onboarding tips.
- **Blurred pop-ups:** context menu per organ (centre, hide, hide others) over a UI blur shader adapted for pop-ups and reused in several apps.
- **Section tool:** cuts the model to show the inside of the organs.
- **Paint tool:** annotate a capture of the model and download the image.
- See-through bone material so the organs stay readable behind it.





05 · SHIPPED

Apps on the stores

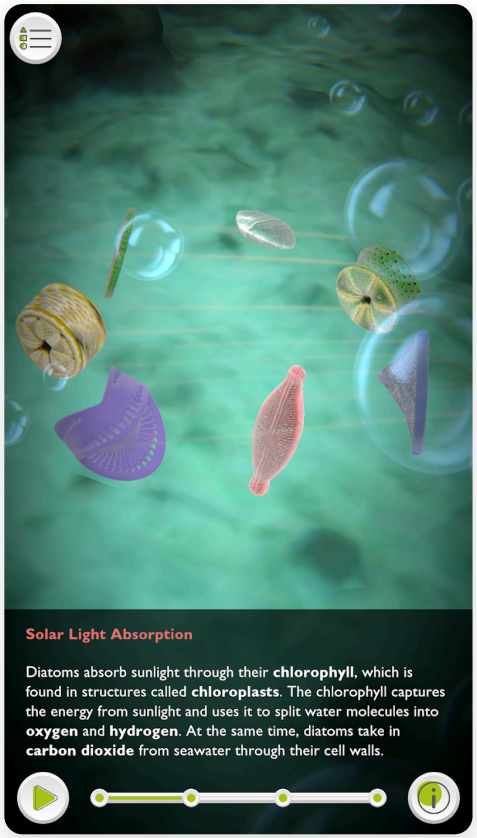
Unity apps I worked on from concept to release: UI, art integration and device performance.



FIAPAS · Colegio Accesible

AR 3D school with 17 interaction points, sound level meter and PDF report.

App Store



Micro Cards AR+

AR companion app for illustrated science cards, with 3D organisms and info panels.

Google Play



Spiromax · BIAL

AR training app for healthcare professionals with a guided demo of the inhaler.

Google Play